AI/ML FRAMEWORKS

PyTorch

A hands-on tour of PyTorch — tensors, autograd, nn.Module, training loops, GPU, checkpoints, and torch.compile — built for real work and interviews.

pytorch · deep-learning · autograd · tensors · gpu · training-loop · torch.compile

Study Pack · Comprehensive reference & interview prep

CONTENTS

1. What PyTorch Is & Why It Matters
2. Setup & Getting Started
3. Tensors: The Core Data Structure
4. Autograd: Automatic Differentiation
5. Building Models with nn.Module
6. Loss Functions & Optimizers
7. Datasets & DataLoader
8. The Training Loop
9. GPU, CUDA & Devices
10. Saving & Loading Checkpoints
11. torch.compile & Faster Inference
12. Worked Mini-Example: XOR in 30 Lines
13. Cheat-Sheet Reference
14. Best Practices & Common Pitfalls
15. Interview Questions & Answers
16. Further Learning

What PyTorch Is & Why It Matters

PyTorch is an open-source tensor and automatic-differentiation library that has become the dominant framework for deep-learning research and, increasingly, production. It feels like NumPy with GPU acceleration and a built-in gradient engine — you write ordinary Python, and PyTorch records what you do so it can differentiate it.

Its killer feature is the **define-by-run** (dynamic) computation graph: the graph is built on the fly as your code executes, so control flow, loops, and conditionals are just normal Python. This is why nearly

every modern model — Transformers, diffusion models, LLMs — ships a PyTorch reference implementation first.

TENSORS

N-dimensional arrays with GPU support and a NumPy-like API.

AUTOGRAD

Automatic reverse-mode differentiation over any tensor computation.

NN & OPTIM

Composable layers, loss functions, and gradient-based optimizers.

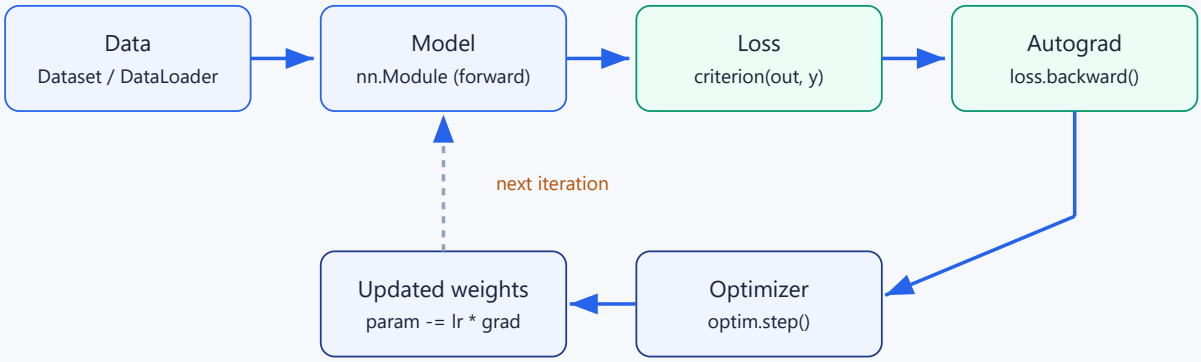


Fig 1. The canonical PyTorch training loop as a data flow.

Mental model

A tensor is data + metadata (shape, dtype, device). When `requires_grad=True`, every operation also records a node in a graph so gradients can flow backward later.

Setup & Getting Started

Install the build matched to your hardware. The official selector at pytorch.org emits the exact command for your CUDA/ROCm/CPU target.

BASH

```
# CPU-only
pip install torch torchvision torchaudio

# CUDA 12.4 build (NVIDIA GPU)
pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/

# Verify
python -c "import torch; print(torch.__version__, torch.cuda.is_available())"
```

PYTHON

```
import torch

# Pick the best available device (CUDA > Apple MPS > CPU)
device = (
    "cuda" if torch.cuda.is_available()
    else "mps" if torch.backends.mps.is_available()
    else "cpu"
)
print(f"Using {device}")
```

> **Tip**
> Pin versions in production. PyTorch, torchvision, and the CUDA build must agree — mismatches are the single most common install failure.

Tensors: The Core Data Structure

Everything in PyTorch is a `torch.Tensor`. The API deliberately mirrors NumPy, so most intuition transfers directly.

PYTHON

```python
import torch

# Creation
a = torch.tensor([[1., 2.], [3., 4.]])   # from data
z = torch.zeros(2, 3)                      # shape (2,3) of zeros
o = torch.ones_like(a)                     # same shape/dtype as a
r = torch.randn(2, 3)                      # standard normal
rng = torch.arange(0, 10, 2)               # [0,2,4,6,8]

# Attributes
print(a.shape, a.dtype, a.device)          # torch.Size([2,2]) torch.float32 cpu

# Reshaping (view shares memory; reshape may copy)
x = torch.arange(12)
x2 = x.view(3, 4)
x3 = x.reshape(2, 6)
x4 = x2.unsqueeze(0)                        # add a batch dim → (1,3,4)
x5 = x4.squeeze(0)                          # remove size-1 dims

# Indexing & slicing (NumPy-style)
print(a[0])        # first row
print(a[:, 1])     # second column
print(a[a > 2])    # boolean mask
```

Broadcasting & Elementwise Math

PYTHON

```python
m = torch.ones(3, 4)
bias = torch.tensor([1., 2., 3., 4.])
print((m + bias).shape)      # (3,4): bias broadcast across rows

# Matrix multiply: @ or matmul; * is elementwise
A = torch.randn(2, 3)
B = torch.randn(3, 5)
C = A @ B                    # (2,5)

# Reductions along axes
t = torch.randn(4, 6)
print(t.sum(dim=0).shape)    # (6,) sum over rows
print(t.mean(dim=1).shape)   # (4,) mean over columns
print(t.argmax(dim=1))       # index of max per row
```

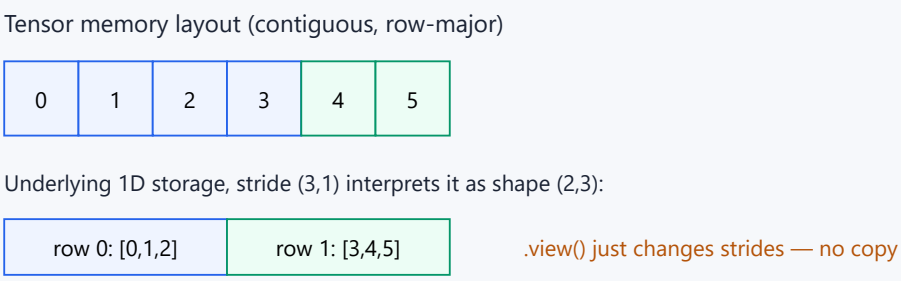


Fig 2. A tensor is a 1D storage buffer plus shape/stride metadata; views reinterpret the same bytes.

> ⚠ **Watch out**
>
> `view()` requires contiguous memory. After operations like `transpose()` you may need `.contiguous()` first, or use `reshape()` which copies when needed.

Autograd: Automatic Differentiation

Autograd is the engine that makes training possible. Set `requires_grad=True` on a tensor and PyTorch records every operation into a dynamic graph. Calling `.backward()` walks that graph in reverse, applying the chain rule to populate each leaf's `.grad`.

PYTHON

```python
x = torch.tensor(2.0, requires_grad=True)
y = x ** 3 + 2 * x          # y = x^3 + 2x
y.backward()                 # dy/dx = 3x^2 + 2 = 14 at x=2
print(x.grad)                # tensor(14.)

# Vector case: provide a gradient (upstream) or reduce to a scalar
w = torch.randn(3, requires_grad=True)
loss = (w ** 2).sum()
loss.backward()
print(w.grad)                # 2*w
```

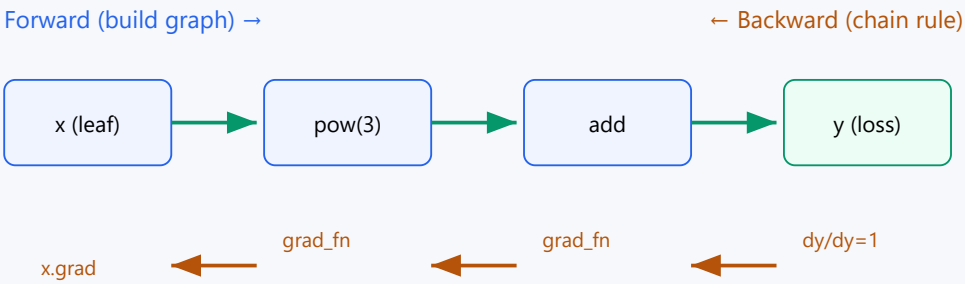


Fig 3. Autograd records each op's `grad_fn` during the forward pass, then applies the chain rule in reverse.

Controlling the Graph

PYTHON

```python
# Disable tracking for inference / updates (saves memory, speeds up)
with torch.no_grad():
    preds = model(x)

# Detach a tensor from the graph (stop gradient)
frozen = tensor.detach()

# Zero grads each step — gradients ACCUMULATE by default!
optimizer.zero_grad()        # or model.zero_grad(set_to_none=True)
```

Gotcha

Gradients accumulate across `.backward()` calls. Forgetting `zero_grad()` is the #1 silent training bug — your updates become sums of stale gradients.

Building Models with nn.Module

`nn.Module` is the base class for all models and layers. You declare submodules in `__init__` and define the computation in `forward`. PyTorch automatically tracks parameters for you.

PYTHON

```python
import torch.nn as nn
import torch.nn.functional as F

class MLP(nn.Module):
    def __init__(self, in_dim, hidden, out_dim):
        super().__init__()
        self.fc1 = nn.Linear(in_dim, hidden)
        self.fc2 = nn.Linear(hidden, out_dim)
        self.dropout = nn.Dropout(0.2)

    def forward(self, x):
        x = F.relu(self.fc1(x))
        x = self.dropout(x)
        return self.fc2(x)          # logits (no softmax here)

model = MLP(784, 256, 10)
print(sum(p.numel() for p in model.parameters()))  # param count
print(model)                                       # pretty structure
```

COMMON LAYERS

`Linear`, `Conv2d`, `LSTM`, `MultiheadAttention`, `Embedding`, `BatchNorm2d`, `LayerNorm`, `Dropout`.

CONTAINERS

`nn.Sequential` for linear stacks, `nn.ModuleList` / `nn.ModuleDict` when you need Python control flow over submodules.

PYTHON

```python
# Quick stacks with Sequential
net = nn.Sequential(
    nn.Linear(784, 256), nn.ReLU(),
    nn.Linear(256, 10),
)

# train() vs eval() changes Dropout / BatchNorm behaviour
model.train()   # dropout active, BN uses batch stats
model.eval()    # dropout off, BN uses running stats
```

⚠ **Don't forget**

`model.eval()` before validation/inference, and `model.train()` before training. Mixing them up produces wrong metrics and unstable BatchNorm.

Loss Functions & Optimizers

PYTHON

```python
import torch.optim as optim

criterion = nn.CrossEntropyLoss()        # expects raw logits + int class labels
# criterion = nn.MSELoss()               # regression
# criterion = nn.BCEWithLogitsLoss()     # binary, numerically stable

optimizer = optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-2)
# optimizer = optim.SGD(model.parameters(), lr=0.1, momentum=0.9)

# Learning-rate schedule
scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=50)
```

Optimizer	When to use	Notes
SGD + momentum	CNNs, when you can tune LR	Often best final accuracy; needs schedule
Adam	Fast prototyping, sparse grads	Robust defaults, adaptive per-param LR
AdamW	Transformers / modern default	Decoupled weight decay — prefer over Adam
RMSprop	RNNs, RL	Non-stationary objectives

Datasets & DataLoader

A `Dataset` maps indices to samples; a `DataLoader` wraps it to provide batching, shuffling, and parallel loading.

PYTHON

```python
from torch.utils.data import Dataset, DataLoader

class CSVDataset(Dataset):
    def __init__(self, X, y):
        self.X = torch.as_tensor(X, dtype=torch.float32)
        self.y = torch.as_tensor(y, dtype=torch.long)

    def __len__(self):
        return len(self.X)

    def __getitem__(self, idx):
        return self.X[idx], self.y[idx]

train_ds = CSVDataset(X_train, y_train)
train_loader = DataLoader(
    train_ds, batch_size=64, shuffle=True,
    num_workers=4,          # parallel CPU workers
    pin_memory=True,        # faster host→GPU transfer
    drop_last=True,
)
```

✦ **Throughput tip**

Combine `num_workers>0`, `pin_memory=True`, and `non_blocking=True` on `.to(device)` to overlap data transfer with compute. On Windows, guard the entry point with `if __name__ == "__main__":` when using workers.

The Training Loop

PyTorch does not hide the loop — you write it. The five steps are always the same: forward, loss, zero grads, backward, step.

PYTHON

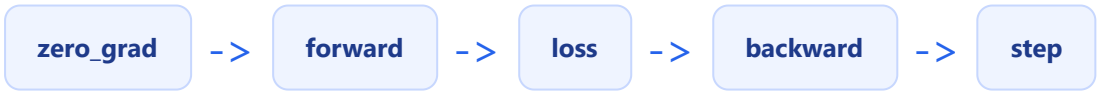
```python
model.to(device)

for epoch in range(num_epochs):
    model.train()
    running = 0.0
    for xb, yb in train_loader:
        xb = xb.to(device, non_blocking=True)
        yb = yb.to(device, non_blocking=True)

        optimizer.zero_grad(set_to_none=True)   # 1. clear old grads
        logits = model(xb)                       # 2. forward
        loss = criterion(logits, yb)             # 3. compute loss
        loss.backward()                          # 4. backprop
        optimizer.step()                         # 5. update weights

        running += loss.item() * xb.size(0)
    scheduler.step()

    # ——— validation ———
    model.eval()
    correct = total = 0
    with torch.no_grad():
        for xb, yb in val_loader:
            xb, yb = xb.to(device), yb.to(device)
            pred = model(xb).argmax(dim=1)
            correct += (pred == yb).sum().item()
            total += yb.size(0)
    print(f"epoch {epoch}: loss={running/len(train_ds):.4f} acc={correct/total:.3f}")
```



Mixed Precision (AMP)

PYTHON

```python
from torch.amp import autocast, GradScaler

scaler = GradScaler("cuda")
for xb, yb in train_loader:
    xb, yb = xb.to(device), yb.to(device)
    optimizer.zero_grad(set_to_none=True)
    with autocast("cuda", dtype=torch.float16):   # or bfloat16
        loss = criterion(model(xb), yb)
    scaler.scale(loss).backward()      # scale to avoid fp16 underflow
    scaler.step(optimizer)
    scaler.update()
```

✓ **Why**

AMP runs most ops in fp16/bf16 for ~2x speed and ~half the memory on modern GPUs, keeping a fp32 master copy for stability.

GPU, CUDA & Devices

PYTHON

```python
# Move model and data to GPU — BOTH must be on the same device
model = model.to("cuda")
x = x.to("cuda")

# Device-agnostic code
x = x.to(device)

# Inspect / manage memory
print(torch.cuda.memory_allocated() / 1e9, "GB")
torch.cuda.empty_cache()           # return cached blocks to the driver

# Multi-GPU (single node, simplest): DataParallel is legacy;
# prefer DistributedDataParallel (DDP) for real workloads
# torchrun --nproc_per_node=4 train.py
```

Symptom	Likely cause	Fix
RuntimeError: Expected all tensors on same device	Model on GPU, data on CPU (or vice versa)	`.to(device)` both
CUDA out of memory	Batch too large / grads retained	Smaller batch, AMP, `no_grad`, grad accumulation
Slow GPU utilization	Data loading bottleneck	More workers, `pin_memory`, prefetch

> ⚠ **OOM tip**
> `loss.item()` or `float(loss)` detaches the scalar. Accumulating `running += loss` (the tensor, not `.item()`) keeps the whole graph alive and leaks memory across the epoch.

Saving & Loading Checkpoints

Save the `state_dict` (a dict of tensors), not the whole model object — it is portable and version-robust.

PYTHON

```python
# Save a full training checkpoint
torch.save({
    "epoch": epoch,
    "model_state": model.state_dict(),
    "optim_state": optimizer.state_dict(),
    "scheduler_state": scheduler.state_dict(),
    "best_acc": best_acc,
}, "checkpoint.pt")

# Restore
ckpt = torch.load("checkpoint.pt", map_location=device, weights_only=True)
model.load_state_dict(ckpt["model_state"])
optimizer.load_state_dict(ckpt["optim_state"])
start_epoch = ckpt["epoch"] + 1
```

Security

As of recent PyTorch, pass `weights_only=True` to `torch.load` for untrusted files — the default was flipped to `True` to block arbitrary-code execution via pickle. Only disable it for files you fully trust.

torch.compile & Faster Inference

Introduced in PyTorch 2.0, `torch.compile` JIT-compiles your model into fused, optimized kernels via TorchInductor — often a large speedup with a one-line change and no code rewrite.

PYTHON

```python
model = torch.compile(model)          # default mode
# model = torch.compile(model, mode="max-autotune")  # longer warmup, faster steady-st

# First call traces & compiles (slow); subsequent calls are fast.
```

Inference-time checklist

PYTHON

```python
model.eval()
with torch.inference_mode():          # like no_grad but faster (no view tracking)
    out = model(x.to(device))
    probs = torch.softmax(out, dim=1)
    pred = probs.argmax(dim=1)
```

TORCHSCRIPT

`torch.jit.script/trace` → serialized, Python-free deployment.

ONNX

`torch.onnx.export` → run in other runtimes (TensorRT, ORT).

QUANTIZATION

int8 weights/activations shrink models and speed up CPU inference.

Worked Mini-Example: XOR in 30 Lines

Show the full runnable example

Cheat-Sheet Reference

Task	Code
To GPU	`t.to("cuda")` / `t.cuda()`
NumPy ↔ Tensor	`torch.from_numpy(a)` / `t.numpy()`
No-grad block	`with torch.no_grad():`
Add/remove dim	`t.unsqueeze(0)` / `t.squeeze()`
Concatenate	`torch.cat([a,b], dim=0)`
Stack new axis	`torch.stack([a,b])`
Scalar out	`t.item()`
Clip grads	`nn.utils.clip_grad_norm_(params, 1.0)`
Count params	`sum(p.numel() for p in m.parameters())`
Seed everything	`torch.manual_seed(0)`

PyTorch vs NumPy vs TensorFlow

Aspect	PyTorch	NumPy	TensorFlow
GPU	Yes	No	Yes
Autodiff	Yes (dynamic)	No	Yes (eager + graph)
Graph	Define-by-run	—	Eager default, tf.function
Research share	Dominant	—	Declining

Best Practices & Common Pitfalls

✓ **Do**

Use `zero_grad(set_to_none=True)`, pass logits to `CrossEntropyLoss` (never pre-softmax), seed RNGs for reproducibility, and move the model to the device *before* constructing the optimizer.

✦ **Do**

Profile with `torch.profiler` and prefer `AdamW` + a cosine schedule as a strong modern default. Checkpoint every epoch and keep the best-by-validation copy.

⚠ **Avoid**

Calling `.cuda()` on data but not the model (device mismatch); accumulating loss tensors instead of `.item()`; forgetting `eval()`; applying softmax before `CrossEntropyLoss`.

✕ **Never**

Load untrusted checkpoints with `weights_only=False`, and never leave `requires_grad=True` on inputs during inference — it silently builds graphs and wastes memory.

Interview Questions & Answers

What is the difference between a dynamic and a static computation graph, and which does PyTorch use?

PyTorch builds a **dynamic** (define-by-run) graph: the graph is constructed as the forward pass executes, so Python control flow works naturally and debugging is easy. Static-graph frameworks define the whole graph first, then execute it — faster to optimize but harder to debug. `torch.compile` now gives PyTorch much of the static-graph performance while keeping the eager programming model.

Why must you call optimizer.zero_grad() each iteration?

Because `.backward()` *accumulates* gradients into `.grad` rather than overwriting them. Without zeroing, each step uses the sum of all previous gradients, corrupting training. (Accumulation is intentional — it lets you simulate larger batches by delaying the `step`.)

What does requires_grad do, and what is a leaf tensor?

`requires_grad=True` tells autograd to track operations on that tensor so gradients can be computed. A **leaf** is a tensor created directly by the user (e.g. model parameters) rather than as the result of an op; only leaves with `requires_grad` accumulate `.grad` after `backward()`.

When would you use torch.no_grad() vs detach() vs torch.inference_mode()?

`no_grad()` is a context that disables graph tracking for everything inside — used during validation. `detach()` returns a single tensor cut off from the graph. `inference_mode()` is like `no_grad` but even faster because it also skips view/version tracking; use it for pure forward-only inference.

Why pass raw logits to CrossEntropyLoss instead of softmax probabilities?

`nn.CrossEntropyLoss` fuses `LogSoftmax` + `NLLLoss` internally for numerical stability (the log-sum-exp trick avoids overflow/underflow). Applying softmax yourself first double-counts the normalization and produces wrong, unstable gradients.

What is the difference between model.train() and model.eval()?

They toggle layer behaviour. In `train()`, Dropout randomly zeros activations and BatchNorm uses per-batch statistics while updating running estimates. In `eval()`, Dropout is a no-op and BatchNorm uses the stored running mean/variance. They do *not* disable gradients — pair `eval()` with `no_grad()`.

How do view() and reshape() differ?

`view()` returns a tensor sharing the same storage and requires the data to be contiguous — it only changes strides. `reshape()` returns a view when possible but silently copies when the layout prevents a view. Use `view()` when you want to guarantee no copy, `reshape()` when you just want the shape.

What causes a CUDA out-of-memory error and how do you reduce memory?

Common causes: batch size too large, holding onto tensors still attached to the graph (e.g. accumulating `loss` instead of `loss.item()`), or retaining the graph with `retain_graph=True`. Fixes: lower batch size, use AMP, gradient accumulation, `torch.no_grad()` for eval, gradient checkpointing, and free references so Python can garbage-collect.

Why save state_dict() rather than the whole model?

A `state_dict` is just an `OrderedDict` of tensors — it is portable, inspectable, and does not pickle your class definitions or file paths. Saving the whole model via pickle breaks if you refactor the class or move modules, and is a code-execution security risk on load.

What does torch.compile do and when should you use it?

> It traces your model with TorchDynamo and compiles fused kernels via TorchInductor, typically giving a sizeable speedup with one line and no rewrite. Use it once your model is correct and stable; expect a slow first iteration (compilation) and watch for recompilations caused by changing input shapes (control with `dynamic=True`).

> **How does DistributedDataParallel differ from DataParallel?**
>
> `DataParallel` runs on one process, splitting a batch across GPUs with a single Python GIL — simple but slow and now discouraged. `DistributedDataParallel` runs one process per GPU, each with its own data shard, syncing gradients via all-reduce. DDP scales far better across GPUs and nodes and is the recommended approach.

> **What is gradient accumulation and why is it useful?**
>
> Instead of calling `optimizer.step()` every batch, you run `backward()` on several small batches (letting grads accumulate) and step once every N batches, dividing loss by N. This simulates a batch N times larger than fits in memory — crucial for training big models on limited GPUs.

Further Learning

- **Official tutorials** — the "Deep Learning with PyTorch: A 60 Minute Blitz" and "Learn the Basics" tracks at pytorch.org/tutorials.
- **Autograd mechanics** — read the official autograd docs to understand the graph, hooks, and custom `autograd.Function`.
- **Ecosystem** — *torchvision*, *torchaudio*, *Hugging Face Transformers*, and *PyTorch Lightning* for less boilerplate.
- **Scaling** — DistributedDataParallel, FSDP (Fully Sharded Data Parallel), and *torchrun* for multi-GPU/multi-node training.
- **Performance** — `torch.profiler`, `torch.compile` modes, CUDA graphs, and quantization/pruning for deployment.
- **Build something** — reimplement a small Transformer or ResNet from scratch; nothing cements the API like writing the training loop yourself.